

네트워크 게임 프로그래밍

6팀 추진 계획서

20211800009 박경준

20211800010 박대원

20211800035 임찬진

목차

1. 애플리케이션 기획
2. HIGH LEVEL DESIGN
3. LOW LEVEL DESIGN
4. 역할 분담
5. 개발 환경
6. 개별 일정

애플리케이션 기획

제목 : Geometry Dash

제작자 : 박대원, 민정원

제작한 수업 : 윈도우 프로그래밍

● 게임 소개

플레이어가 장애물을 피해 끝인 지점까지 도달하는 것이 목표인 게임이다.

플레이어의 동작 버튼이 Space Bar 하나로 간단하며 게임 내부에서는 다양한 타일들이 존재합니다.

플레이어는 직접 맵을 제작할 수 있어 이용자의 흥미를 둘 수 있습니다.

● 기존에 있던 기능

싱글플레이 모드

● 새로 추가할 기능

1vs1vs1 모드

● 키 입력 방식

◎ 실제 게임 플레이에서 사용

- 일반 모드

Space Bar를 눌러 점프가 가능합니다.

2단 점프 블록과 충돌 시 타이밍에 맞게 Space Bar를 누르면 공중에서 점프가 가능합니다.

- 비행 모드

Space Bar를 누르고있으면 위로 올라갑니다.

Space Bar를 떼면 내려갑니다.

- 지그재그 모드

Space Bar를 누르고있으면 위로 올라갑니다.

Space Bar를 떼면 내려갑니다.

◎ 모든 상황에서 사용

각 단계에서는 ESC를 눌러 지도나 메인화면으로 나갈 수 있습니다.

● 각 씬 설명

◎ 메인 메뉴 씬

게임의 대표 메인메뉴 입니다. 구성은 제목과 플레이 버튼입니다.

◎ 레벨 선택 씬

어떤 게임 모드를 플레이 할지에 대한 선택 버튼입니다. 2개의 모드가 있으며 각각

1. 기본 게임 씬
2. 1vs1vs1 씬

위 2가지 모드를 플레이 할 수 있는 버튼을 제작해줄 예정입니다.

◎ 1vs1vs1 씬

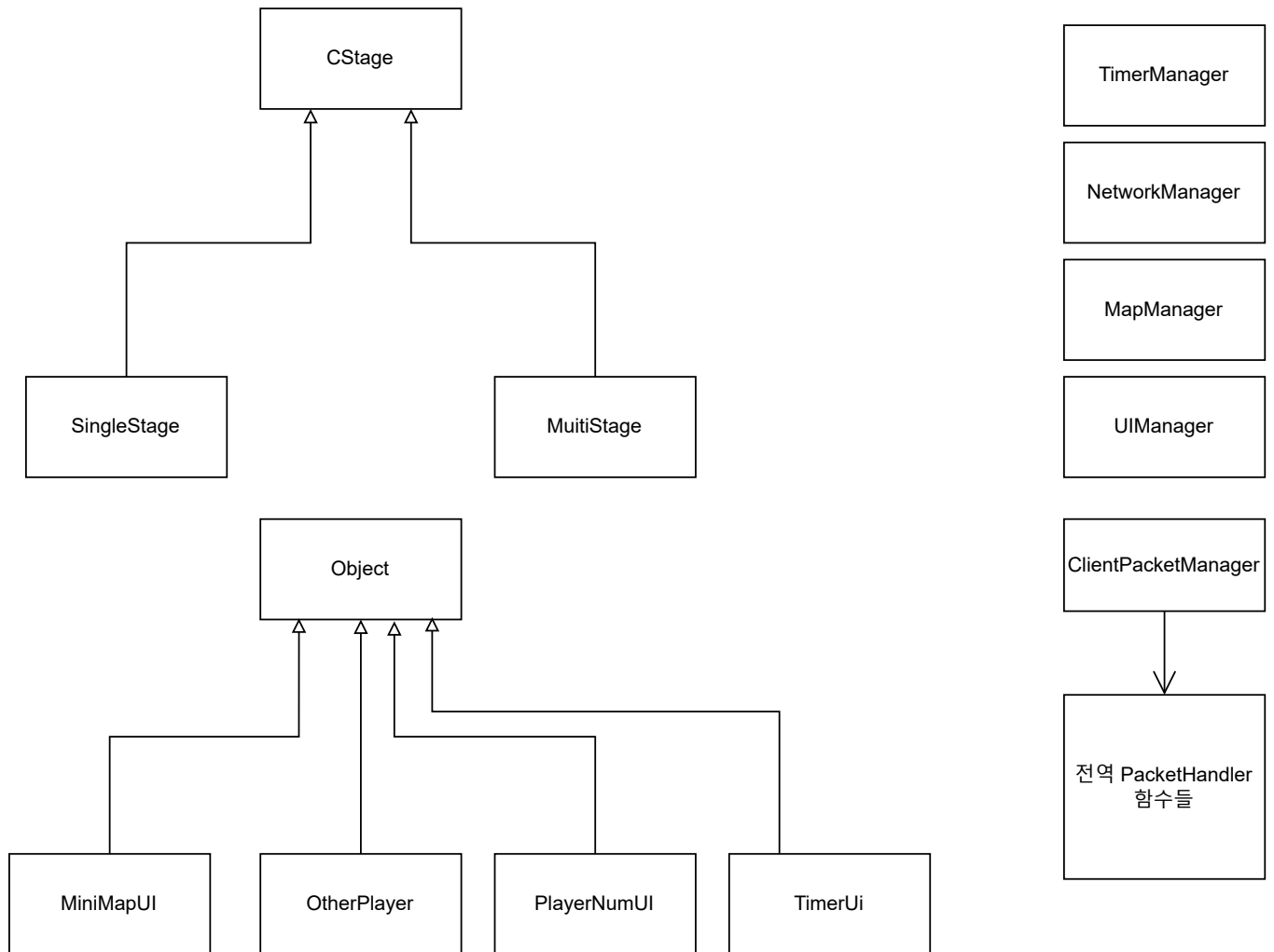
3명의 플레이어가 제작자가 미리 제작해놓은 맵 중 랜덤한 맵을 하나 플레이하며 가장 먼저 죽지않고 결승선에 도착한 플레이어가 우승하게되는 게임입니다.

화면에는 자신 뿐만 아니라 모든 사람들의 플레이어가 함께 달리도록 제작할 예정입니다.

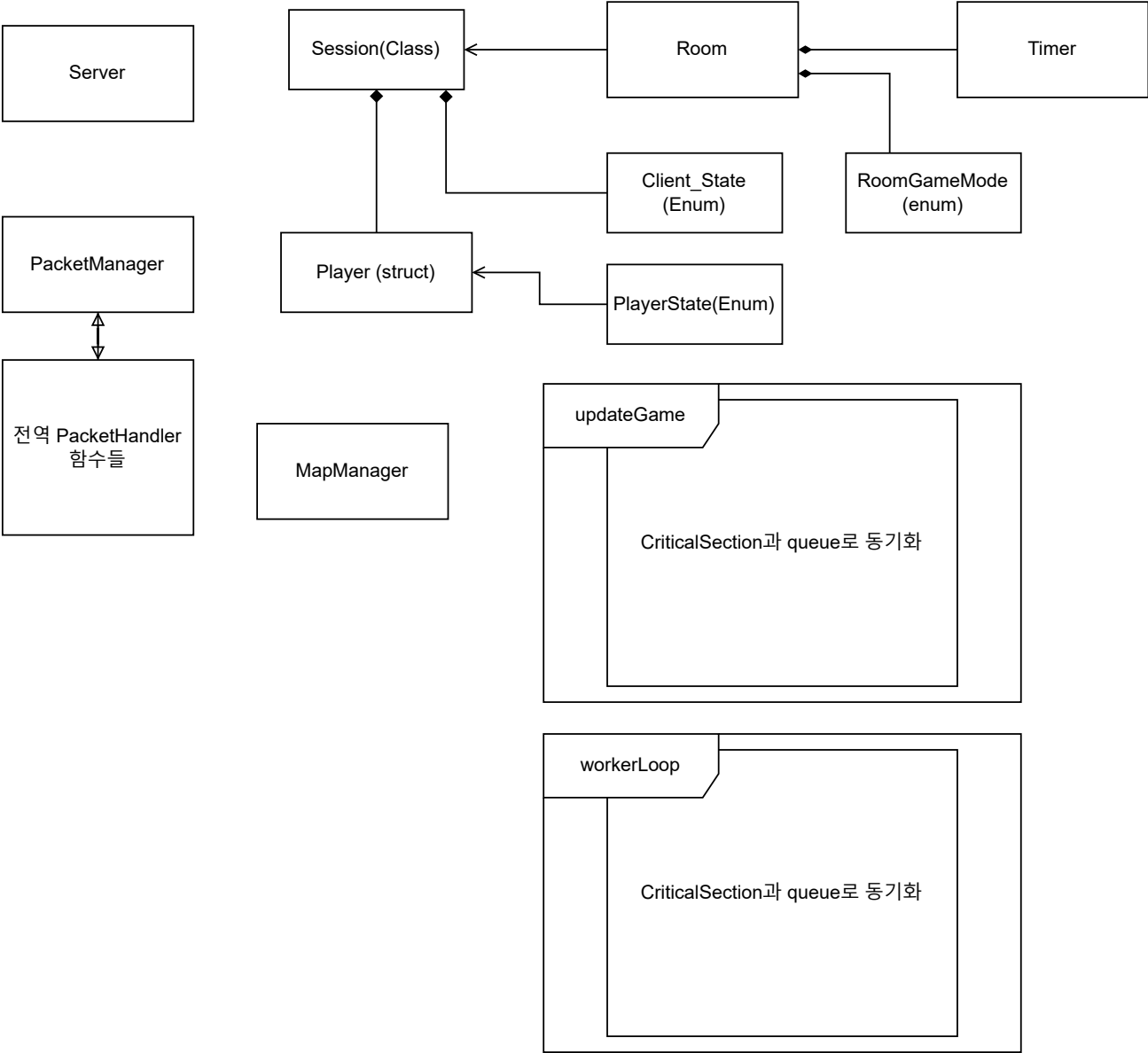
- 화면 구성

우측 상단에는 시간, 중앙 상단에는 미니맵을 띄워 해당 맵의 길이에 대한 모든 플레이어들의 진행 상황을 확인할 수 있도록 제작할 예정입니다.
그리고 좌측 상단에는 플레이어의 번호를 띄워줄 예정입니다.

Client High Level Design



Server High Level Design



Client Low Level Design

Stage

```
Stage();  
virtual ~Stage();  
  
virtual void Initailizer() = 0;  
  
virtual void Update(float fTime) = 0;  
  
virtual void LateUpdate(float fTime) = 0;  
  
virtual void Render(HDC mDC) = 0;  
  
virtual void Free() = 0;
```

SingleStage : Stage

```
SingleStage();  
virtual ~SingleStage();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

MultiStage : Stage

```
MultiStage();  
virtual ~MultiStage();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

Object

```
Object();  
virtual ~Object();  
  
virtual void Initailizer() = 0;  
  
virtual void Update(float fTime) = 0;  
  
virtual void LateUpdate(float fTime) = 0;  
  
virtual void Render(HDC mDC) = 0;  
  
virtual void Free() = 0;  
  
SetPosRect();  
GetPosRect();  
  
SetCenterPos();  
GetCenterPos();  
  
ResetPosRect();  
  
SetRidius();  
  
GetRidius();  
  
SetDead();  
  
변수  
bool m_bDead{};  
RECT m_POSRECT{};  
POINT m_CenterPos{};  
float m_fRidius{};
```

MiniMapUI : Object

```
MiniMapUI();  
virtual ~MiniMapUI();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

PlayerNumUI : Object

```
PlayerNumUI();  
virtual ~PlayerNumUI();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

OtherPlayer : Object

```
OtherPlayer();  
virtual ~OtherPlayer();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

PlayerNumUI : Object

```
PlayerNumUI();  
virtual ~PlayerNumUI();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

<u>TimerManager</u>
GetInstance();
DestoryInstance();
GetDeltaTime();
GetFPS();
Reset();
Tick();
GetTotalTime();

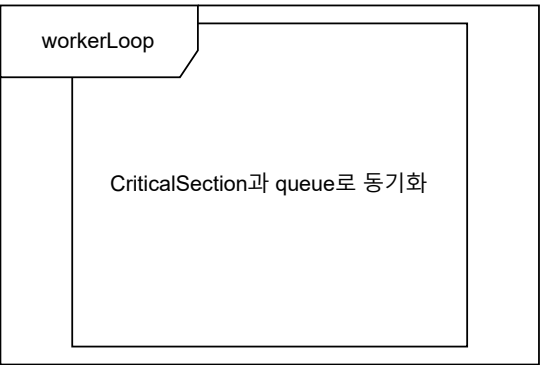
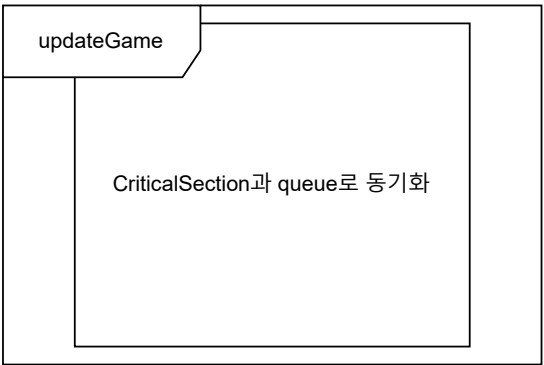
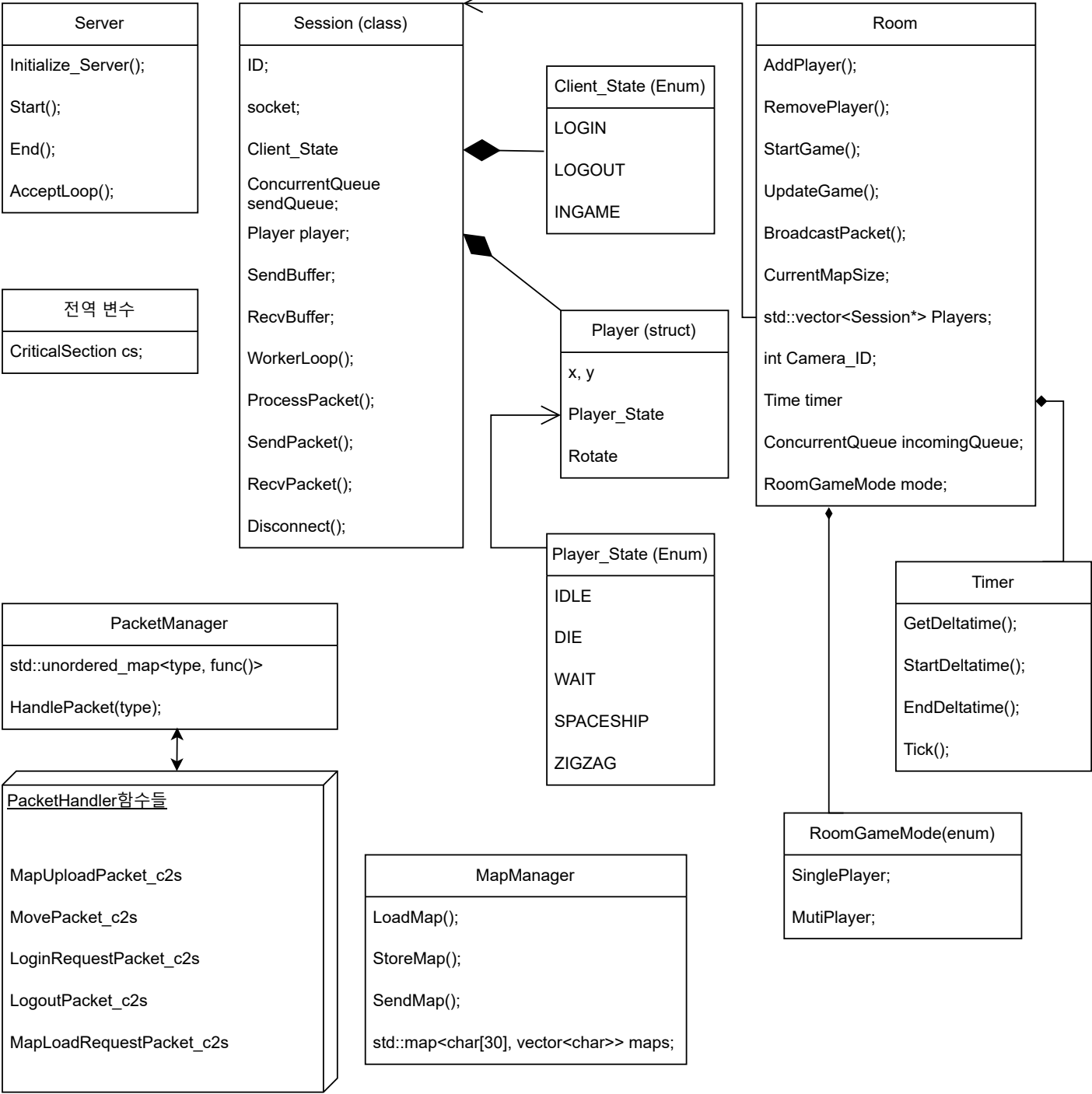
<u>MapManager</u>
GetInstance();
DestoryInstance();
MapShow();
MapLoad();
MapStage();

<u>NetworkManager</u>
GetInstance();
DestoryInstance();
initialize_Client();
sendPacket();
recvPacket();
UpdatePacket();
Shutdown();

<u>UIManager</u>
GetInstance();
DestoryInstance();
MiniMapUpdate();
MiniMapShow();
ScoreBoardShow();
TimerShow();
UserNumShow();

<u>전역 핸들러 함수</u>
MovePacket_s2c
LoginAcceptPacket_s2c
MapNamesPacket_s2c
MapLoadPacket_s2c
RoomEnterAcceptPacket_s2c
RoomStartPacket_s2c
MapUploadDonePacket_s2c
MapRoomEndPacket_s2c

Server Low Level Design



Packet

Client => Server

PacketManager
std::unordered_map<PacketType, func()> Handlers; RegisterHandler(type, func()); Handle(type,session);

Sever => Client

PacketManager
std::unordered_map<PacketType, func()> Handlers; RegisterHandler(type, func()); Handle(type);

MovePacket
long long size; PacketType type float x,y; int id float rotate int cameraID; Player_State player_state; float minMapPercentage;

MapNamesPacket
long long size; PacketType type std::array<char[30], 10> mapNames

MapLoadPacket
long long size; PacketType type char[30] mapNames vector<char> mapData;

LoginAcceptPacket
long long size; type int id

RoomEnterAcceptPacket
long long size; PacketType type
RoomStartPacket
long long size; PacketType type

MovePacket
long long size; PacketType type float x,y; float rotate Player_State player_state; float x,y;

LoginReqeustPacket
long long size; type

LogoutPacket
long long size; PacketType type

MapLoadRequestPacket
long long size; PacketType type char[30] mapName

RoomEnterPacket
long long size; PacketType type int roomID;

MapUploadPacket
long long size; PacketType type char[30] mapName; vector<char> mapData;

MapUploadDonePacket
long long size;
PacketType type

RoomEndPacket
long long size;
PacketType type
int roomID;
int winUserID
float ElapsedTime

RoomLeavePacket
long long size;
PacketType type
int roomID;

PacketType(enum class)
MovePacket_s2c
MovePacket_c2s
LoginRequestPacket_c2s
LoginAcceptPacket_s2c
LogoutPacket_c2s
MapNamesPacket_s2c
MapLoadRequestPacket_c2s
MapLoadPacket_s2c
RoomEnterPacket_c2s
RoomEnterAcceptPacket_s2c
RoomStartPacket_s2c
MapUploadPacket_c2s
MapUploadDonePacket_s2c
MapRoomLeavePacket_c2s
MapRoomEndPacket_s2c

역할 분담

박경준	● 서버 & 세션 (I/O) 담당, Server 클래스와 Session의 핵심 I/O(네트워크 송수신)를 담당합니다.
박대원	● Room & Game Logic 담당 Room 클래스와 게임 로직을 처리하는 UpdateGame를 담당합니다.
임찬진	● 서버의 유틸리티 & 핸들러 (Data & Packet) 담당 PacketManager, Timer, MapManager 등 유틸리티 클래스와 개별 패킷 핸들러 함수 구현을 담당합니다.

개발 환경

개발 도구	visual studio 2022
버전 관리	github
개발 언어	C++ STL
API	WINDOWS, WINSOCK2

박경준 개별 일정

		25/10/28	25/10/29	25/10/30	25/10/31	25/11/01
						Server 멤버함수 Initialize _Server(), Start(), End() 구현
25/11/02	25/11/03	25/11/04	25/11/05	25/11/06	25/11/07	25/11/08
Server:: AcceptLoop(), Session 클래스 기본 구조 설계		workerLoop, Session::Disco nnect()	Room의 AddPlayer RemovePlayer			
25/11/09	25/11/10	25/11/11	25/11/12	25/11/13	25/11/14	25/11/15
		workerLoop 내 RecvPacket 구현	CriticalSection RecvBuffer 패킷 역직렬화, imcomingQue ue.push(packe t)			
25/11/16	25/11/17	25/11/18	25/11/19	25/11/20	25/11/21	25/11/22
		workerLoop 내 SendPacket 구현	CriticalSection sendQueue.po p()/send(), SendLeftoverB uffer			
25/11/23	25/11/24	25/11/25	25/11/26	25/11/27	25/11/28	25/11/29
		Client_State 관리 연동	AcceptLoop Session::work erLoop 부하 테스트, SingleStage, MultiStage 구현			
25/11/30	25/12/01	25/12/02	25/12/03	25/12/04	25/12/05	25/12/06
		동시 접속/종료 예외 처리, 소켓 오류 핸들링	Session 인터페이스 정리/리팩토링, 서버 부하 테스트			
25/12/07	25/12/08	25/12/09	25/12/10	25/12/11	25/12/12	
	버그 수정, Server::End() 자원 해제 점검	최종 코드 점검 및 시연 준비				

박대원 개별 일정

		25/10/28	25/10/29	25/10/30	25/10/31	25/11/01
						Room 클래스 기본 구조, Player (struct) 정의
25/11/02	25/11/03	25/11/04	25/11/05	25/11/06	25/11/07	25/11/08
UpdateGame 루프			Room멤버함수 AddPlayer, RemovePlayer Room::Broadc astPacket()	Timer 클래스 구현, Room에 Timer 연동		
25/11/09	25/11/10	25/11/11	25/11/12	25/11/13	25/11/14	25/11/15
			PacketManager 연동, LoginRequest Packet_c2s 핸들러, MapManager 기본 구조	MapManager 멤버함수 LoadMap(), 방 시작		
25/11/16	25/11/17	25/11/18	25/11/19	25/11/20	25/11/21	25/11/22
			RoomGameMo de(enum)적용, MapUploadPa cket_c2s 핸들러	Room::Broadc astPacket() 핸들러: MapNamesPac ket_s2c MapLoadPack et_s2c MapLoadRequ estPacket_c2s		
25/11/23	25/11/24	25/11/25	25/11/26	25/11/27	25/11/28	25/11/29
			Room:: StartGame(), 핸들러: RoomStartPac ket_s2c	Player_State 동기화 로직, BroadcastPac ket 구현		
25/11/30	25/12/01	25/12/02	25/12/03	25/12/04	25/12/05	25/12/06
			MutiPlayer 생성	카메라: 관전로직		

25/12/07	25/12/08	25/12/09	25/12/10	25/12/11	25/12/12	
RoomEndPack et, LogoutPacket _c2s 핸들러	버그 수정, 시연 준비					

임찬진 개별 일정

		25/10/28	25/10/29	25/10/30	25/10/31	25/11/01
25/11/02	25/11/03	25/11/04	25/11/05	25/11/06	25/11/07	25/11/08
s2c 패킷 생성, c2s 패킷 생성			PacketType 생성		싱글톤 템플릿 구현 TimerManager, MapManager, UIManager	
25/11/09	25/11/10	25/11/11	25/11/12	25/11/13	25/11/14	25/11/15
NetworkMana ger, ClientPacketM anager 클래스 생성			Object 클래스 구현, NetworkMana ger::initialize_ Client() 구현	NetworkMana ger:: sendPacket(), LoginRequest Packet_c2s 전송		
25/11/16	25/11/17	25/11/18	25/11/19	25/11/20	25/11/21	25/11/22
			핸들러: LoginAcceptP acket_s2c, SingleStage, MultiStage 구현	NetworkMana ger::RecvPack et		
25/11/23	25/11/24	25/11/25	25/11/26	25/11/27	25/11/28	25/11/29
			핸들러: RoomEnterAc ceptPacket_s2 c, MovePacket_c 2s RoomEnterPa cket_c2s	per 50ms 위치전송: MovePacket_c 2s, OtherPlayer 클래스 구현		
25/11/30	25/12/01	25/12/02	25/12/03	25/12/04	25/12/05	25/12/06
			핸들러: MovePacket_s 2c (OtherPlayer 갱신)	UI 추가: PlayerNumUI, MiniMapUI TimerUi		

25/12/07	25/12/08	25/12/09	25/12/10	25/12/11	25/12/12	
UIManager 버그 체크 핸들러: MapRoomEnd Packet_s2c	버그 수정, 썬 전환/카메라 버그 수정, 시연 준비					